

■ 오늘의 지도

네 시간, 하나의 이야기

1교시 · 언어

"에이전트"라는 말, 차분히 풀어보기

Agent · Connector · Plugin · Skill

2교시 · 엔진

모델과 플랫폼을 고르고, 내 일을 넣기

프론티어 선별 · 하네스 이해 · 업무 설계

3교시 · 작업대

그 모델을 어떤 도구에서 부리는가

Claude Desktop vs Claude Code

4교시 · 손

직접 만들어보고 퇴근한다

내 업무 Skill 1개 + Connector 연결

1

언어

요즘 어디서나 들리는 "에이전트"라는 말 —
그 뜻을 차분히 풀어보는 것에서 시작합니다.

■ 2022년 겨울

"이게 무슨 AI냐"

ChatGPT가 처음 나왔을 때를 기억하십니까.

말장난이나 한다고, 계산도 틀린다고 웃음거리가 됐습니다.

그 물건을 만드는 데 수조 원이 들었습니다.

■ 에이전트의 진짜 정의

목표를 향해, 스스로 반복하는 것



신입사원에게 "경쟁사 조사해 와"라고 하면 — 검색하고, 보고, 부족하면 다시 찾습니다.

이 루프가 에이전트입니다.

ReAct, Plan & Execute 같은 기법 이름은 몰라도 됩니다 — 즉흥형·계획형·분업형 일하기 방식 정도면 충분합니다.

■ 좋은 소식

에이전트는 이미 만들어져 있습니다.

Claude Desktop, Claude Code —

오늘 노트북에 설치할 수 있는 완성품입니다.

■ 모델만큼 중요한 것

에이전트의 품질 = 모델 × 설계

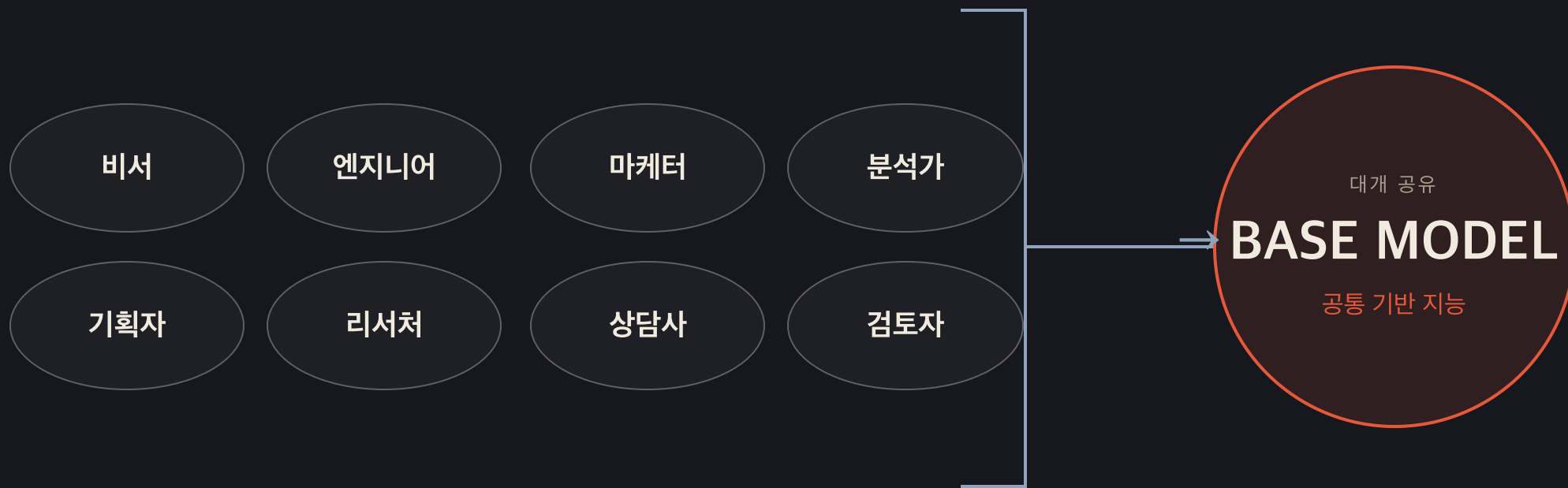
설계란 — 지시를 얼마나 넣고 얼마나 **덜어낼지**,
관찰을 몇 번 시킬지, 얼마나 믿고 맡길지.

Claude Code에 다른 회사 모델을 끼우면 원래 성능이 안 나옵니다.
같은 엔진도 차체가 다르면, 그 차가 아닙니다.

이 이야기는 2교시에서 "하네스"라는 이름으로 다시 만납니다.

■ 먼저 구분할 것 · 역할 수 ≠ 모델 수

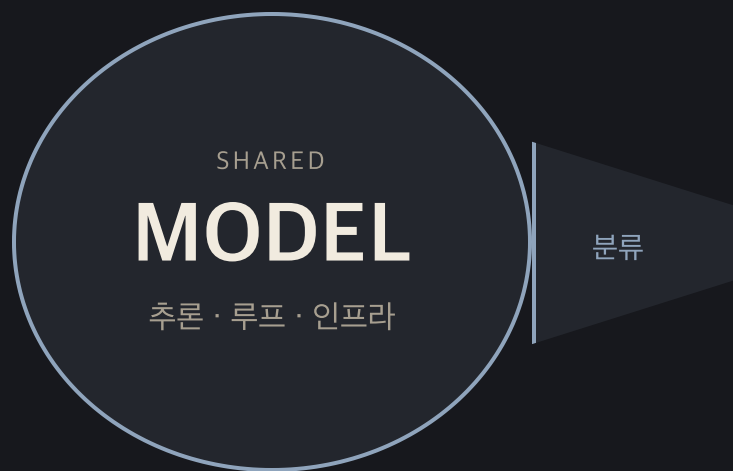
Agent 100개는 맞습니다.
다만 **AI 두뇌 100개**는 아닙니다



서로 다른 직원처럼 일할 수 있습니다. 다만 Agent의 수와 기반 지능의 수는 별개입니다.

■ 공유 엔진 · 서로 다른 직능

같은 엔진 위에서도, 서로 다른 Agent로 일할 수 있습니다



AI 비서

비서 Skill · 기억 · 캘린더 권한

AI 엔지니어

개발 Skill · 저장소 · 테스트 도구

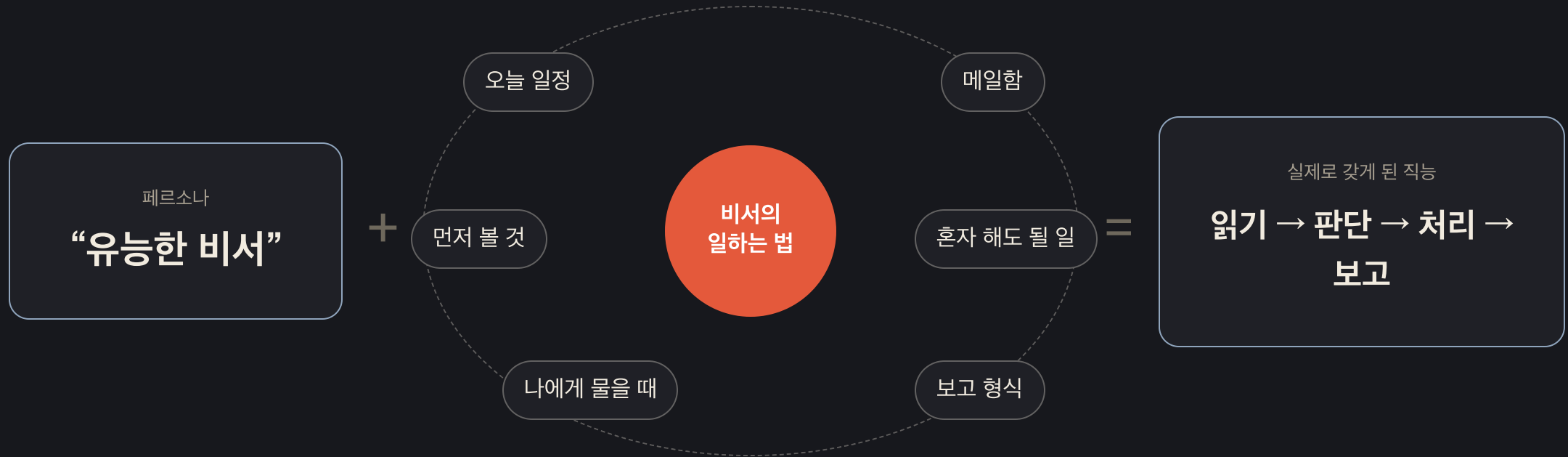
AI 마케터

마케팅 Skill · 브랜드 · 고객 데이터

역할에 Skill · 기억 · 자료 · 도구가 더해지면, 서로 다른 직능을 가진 Agent가 됩니다.

■ 다만, 페르소나만으로는 부족합니다

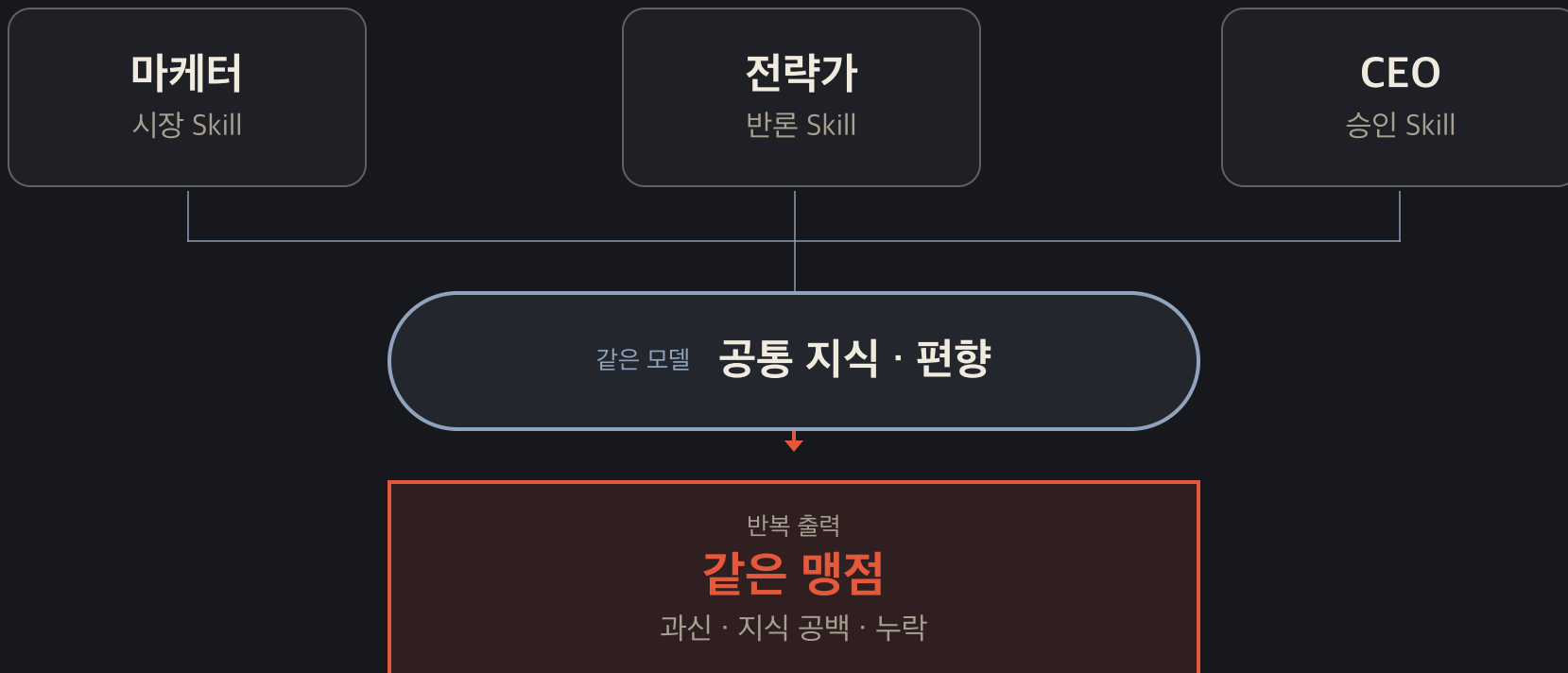
“너는 유능한 비서야”만으로는, 비서의 일을 할 수 없습니다



이 구체적인 ‘일하는 법’을 문서로 적으면 Skill이 됩니다.

■ 공유 모델이 남기는 한계

서로 다른 Skill을 쥐도, 공통 맹점은 남을 수 있습니다



Skill이 직능을 나누고, 독립 자료 · 다른 기준 · 외부 검증이 공통 맹점을 줄입니다.

■ 그래서 1인 회사의 정확한 구조

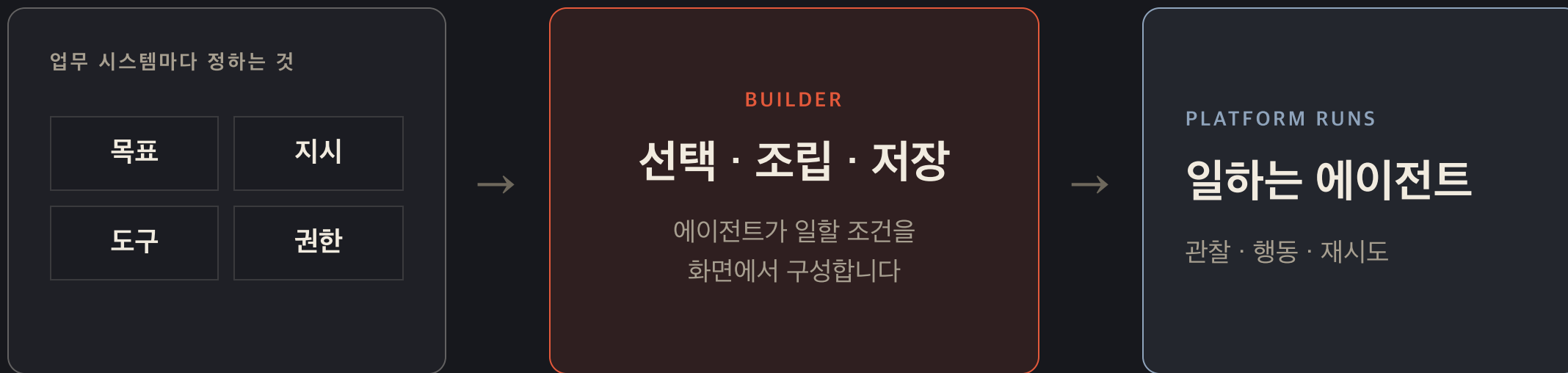
AI 직원 N명은, 업무 시스템 N개로 만들어집니다



직원은 이름표가 아니라 **Skill · 데이터 · 도구 · 검증**의 조합으로 만들어집니다.

■ 그렇다면, 빌더는?

에이전트의 **설정** 작업대입니다



빌더는 실행 주체가 아닙니다. 업무 시스템의 조건을 플랫폼의 실행기에 건네는 인터페이스입니다.

■ 이제 빌더 안을 다시 보면

빌더 안에서 우리가 하는 일은, 사실 **Skill**을 만드는 일입니다

우리가 하는 일

지시문을 쓰고, 도구를 고른다

목표·절차·판단 기준을 적는 일 — 이것이 Skill의 핵심 재료입니다

플랫폼이 하는 일

관찰 · 재시도 · 오류 대처 — 루프 전체

이 부분은 빌려 쓰는 것이라, 플랫폼을 떠나면 함께 반납됩니다

빌더에 흩어져 입력한 **지시 · 절차 · 판단 기준**을 문서로 꺼내 정리하면, 재사용 가능한 Skill이 됩니다.

일하는 에이전트는 곱셈의 결과입니다

내가 쓴 문서

Skill · 역할 정의
절차 · 판단 기준 · 도구 목록

내 것 — 이식 · 상속

×

빌린 실행력

루프 · 재시도 · 위임
(플랫폼의 하네스)

구독 — 계약 종료 시 반납

=

일하는 에이전트

목표를 향해 관찰하고
재시도하는 실행 인스턴스

둘이 결합된 순간에만 존재

참

"그것은 에이전트다" — 결과물을 가리키는 말

절반의 참

"내가 에이전트를 만들었다" — 곱셈의 왼쪽
항만 말한 문장

■ 그래서 "1인 1에이전트"는

한 문장이 아니라, 두 문장의 겹침입니다

"1인 1에이전트"

사용의 독해

참

"1인당 에이전트 하나씩을 부린다"

1인 1PC와 같은 문장. 이미 실현 가능하고, 바람직한 미래입니다.

소유의 독해

확인 필요

"1인당 에이전트 하나씩을 만들어 소유한다"

계약이 끝나면 함께 사라지는 것을 소유라 부르긴 어렵습니다. 남은 것은 문서입니다.

같은 말이 두 뜻을 오갈 때 — 왼쪽을 보고, 오른쪽에 서명하게 되기 쉽습니다

■ 틀린 말일까요?

"내 사무실"은 틀린 말이 아닙니다

대화에서

"내 사무실로 오세요"

임대 사무실이어도 다들 이렇게 말합니다. 자연스러운 말입니다.

등기부등본에서

"내 소유의 사무실"

문서에 적는 순간 이야기가 달라집니다. 건물주는 따로 있습니다.

"1인 1에이전트"도 같습니다 — 구호로는 자연스럽지만,
예산·자산·인수인계 계획서에 적히는 순간 소유의 문장이 됩니다.

■ 구호 완성하기

1인 1에이전트 (사용)

1인 N Skill (소유)

1인 1Agent는 독립 두뇌의 개수가 아니라 업무 창구의 개수입니다.

역할별 Skill이 그 창구를 실제 업무 시스템으로 바꿉니다.

■ 금융사라면 미리 알아둘 것

시연과 도입 사이에는 간격이 있습니다

- 시연은 가장 좋은 모델 + 거기에 맞춘 설계로 이뤄집니다.
- 실제 도입되는 사내 모델이 같은 수준이라는 보장은 없습니다.
- 그런데 내부는 블랙박스라, 도입 후에야 알게 됩니다.

누구의 잘못도 아닌 구조의 문제입니다 — 그래서 질문을 준비해 가는 겁니다.

■ 가져가실 것

벤더에게 드릴 세 가지 질문

질문 ①

시연에 쓴 모델과 우리가 도입
할 모델이 같습니까?

질문 ②

우리 모델 기준으로 최적화됐
다는 근거는 무엇입니까?

질문 ③

이 플랫폼을 떠나면, 우리가 만
든 것 중 무엇이 남습니까?

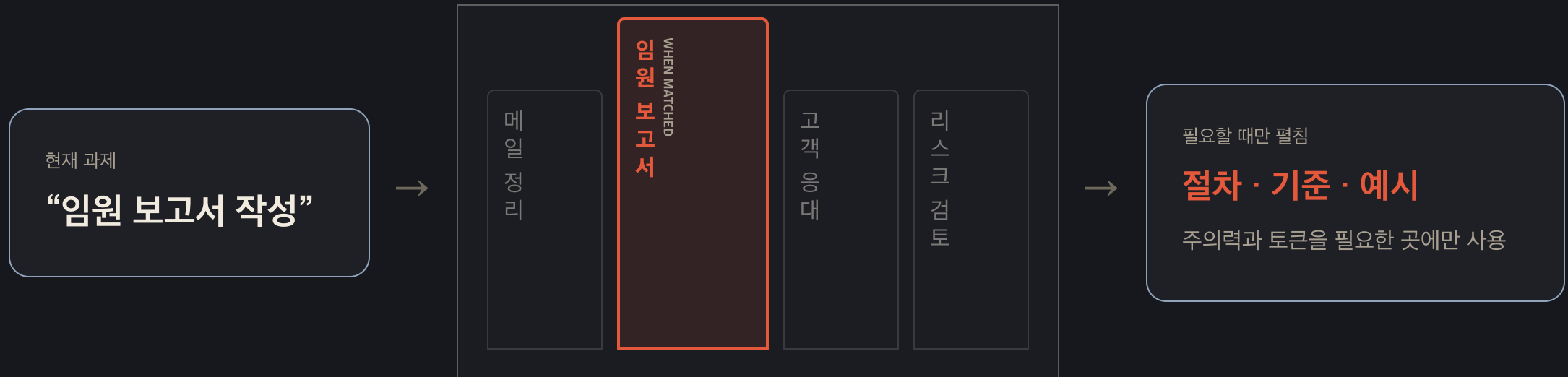
세 번째 질문의 답이 — 오늘 강의의 전부입니다.

플랫폼을 떠나도 남는 것, 어디로든 이식 가능한 문서 — Skill

내 업무 노하우를 AI가 읽을 수 있는 표준업무절차서로 쓴 것.
파일이니까 내 것이고, 옮길 수 있고, 물려줄 수 있습니다.

■ Skill의 실체

특별한 기술이 아니라는 것, 그 자체가 설계입니다



규정집을 통째로 외우지 않고, **필요한 페이지를 필요한 순간에 펼칩니다.**

■ 역사적 검증

엇갈린 반응에서 표준까지, 5개월

2025.10.16	Anthropic, Skill 발표. 반응은 갈렸다 — "그냥 텍스트 아니냐" vs "MCP보다 큰 물건"(사이먼 윌리슨)
2025.12.18	오픈 표준으로 공개
+48시간	Microsoft · OpenAI가 채택 — 경쟁사들이 서로의 형식을 그대로 읽기 시작
2026.03	Google 포함 32개 이상의 도구가 같은 파일을 읽는다 — 사실상의 산업 표준

■ 왜 "여러분의" 자산인가

빌더 안의 설정 vs 내 문서

	노코드 빌더 안의 설정	문서로 관리하는 Skill
소유	플랫폼 계약과 함께 종료	내 파일 — 영구 소유
이동	플랫폼 안에서만	경쟁사 AI들도 같은 파일을 읽음
인수인계	계정 이관 문제	파일 전달 = 노하우 전달

1년 전엔 "한 회사에서만 되는 기능 아니냐"고 할 수 있었습니다. 지금은 아닙니다.

오늘 나오는 말은 네 개뿐입니다

용어	한 줄 정의	회사 비유	오늘
Skill	AI에게 주는 재사용 가능한 업무 지침서 (문서)	표준업무절차서	직접 만든다
Connector	AI가 외부 시스템에 접근하는 표준 연결 통로	시스템 권한 있는 창구	직접 연결한다
Plugin	Skill·설정·연결을 묶어 배포하는 패키지	팀 표준 업무 키트	개념만
Agent	위 재료로 루프를 도는 실행 주체	일 위임받은 주니어	이미 준비되어 있다

Connector는 MCP라는 기술 규격의 소비자용 이름입니다 — 외우실 필요 없습니다, 유인물에 있습니다.

■ 1교시 요약

"에이전트는 이미 준비되어 있다.
우리가 만들 것은,
에이전트에게 줄 일하는 법 — Skill이다."

■ 쉬는 시간 · 10분

10:00

다음 · 2교시 엔진 — 모델과 플랫폼을 고르고, 내 일을 넣기

R 키 — 카운트다운 다시 시작

2

엔진

좋은 모델과 플랫폼을 고르고,
그 안에 내 업무의 일하는 법을 넣습니다.

모델을 고를 때, 세 탭만 엽니다

01 · 전체 지형

Artificial Analysis

지능 · 가격 · 출력 속도 · 응답 지연 · 컨텍스트를 한 화면에서 비교합니다.

“후보군에 누가 들어오는가?”

[leaderboard 열기 ↗](#)

02 · 앱 만들기

DeepSWE

실제 저장소의 긴 코딩 과제에서 성공률 · 비용 · 토큰 · 스텝을 봅니다.

“끝까지 만들 수 있는가?”

[leaderboard 열기 ↗](#)

03 · 한국어 실무

Dnotitia 한국어 리더보드

한국어 추론 · RAG · 도구 호출을 비공개 평가 세트로 확인합니다.

“한국어 업무에서도 버티는가?”

[leaderboard 열기 ↗](#)

먼저, 프론티어 후보를 다섯 개로 좁힙니다

모델	Intelligence	혼합 가격 / 1M	출력 속도
Claude Fable 5	60	\$7.70	65 tok/s
GPT-5.6 Sol	59	\$4.35	55 tok/s
Claude Opus 4.8 [max]	56	\$3.85	58 tok/s
GPT-5.6 Terra [max]	55	\$2.17	136 tok/s
Claude Sonnet 5 [max]	53	\$1.54	86 tok/s

Artificial Analysis Models Leaderboard · 2026-07-21 확인. 숫자는 결론이 아니라 다음 시험에 올릴 후보를 고르는 필터입니다.

긴 코딩 과제에서는 순서가 다시 바뀝니다

모델 [생각 강도]	성공률	평균 비용	스텝 수
gpt-5.6-sol [max]	73%	\$8.39	61
claude-fable-5	70%	\$21.63	88
gpt-5.6-terra [max]	70%	\$4.95	76
claude-opus-4.8 [max]	59%	\$13.22	120
claude-sonnet-5 [max]	54%	\$26.40	268

91개 저장소 · 113개 장기 과제 · 5개 언어 · 16개 모델 (DeepSWE, 2026-07-21 확인)

■ 교차해서 읽기

같은 모델도, 과제가 바뀌면 경제성이 바뀝니다

GPT-5.6 TERRA

균형 후보

전체 지형	55 · \$2.17
긴 코딩	70% · \$4.95
스텝	76

CLAUDE SONNET 5

조건 확인

전체 지형	53 · \$1.54
긴 코딩	54% · \$26.40
스텝	268

CLAUDE OPUS 4.8

비교 후보

전체 지형	56 · \$3.85
긴 코딩	59% · \$13.22
스텝	120

일반 토큰 가격이 낮아도, 특정 과제에서 오래 헤매면 총비용은 높아집니다. 단가가 아니라 **완료 한 건의 비용**을 봅니다.

영어 시험을 통과해도, 한국어 업무는 다시 봅니다

30

한국어 평가 문항

비공개 평가 세트

추론

한국어 지시와 맥락을 따라 결론에 도달하는가

RAG

주어진 근거를 찾아 답에 정확히 반영하는가

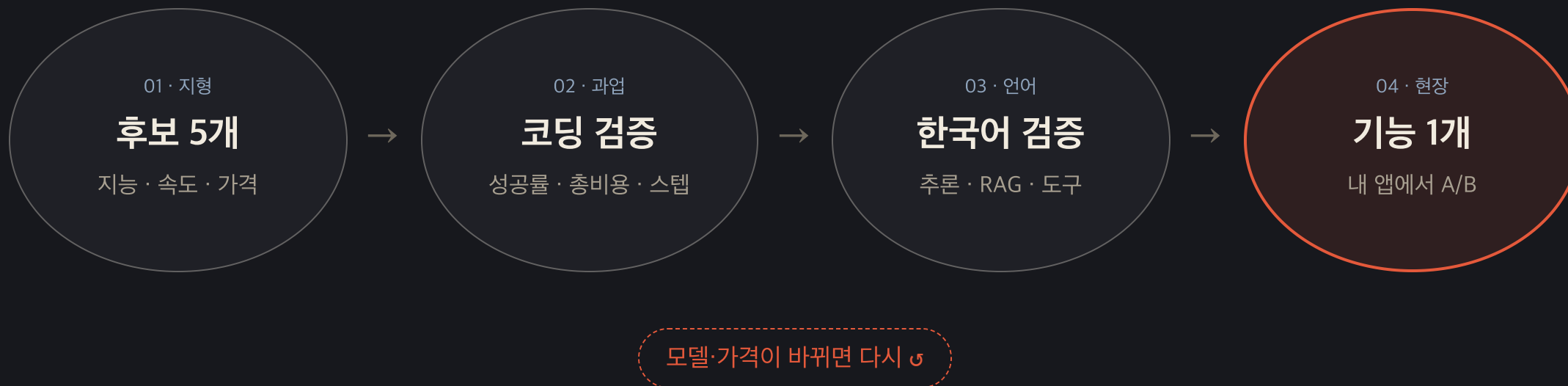
도구 호출

필요한 도구와 인자를 올바르게 선택하는가

채점

문항별 0-1 평균 · 0.7 이상을 정답으로 집계

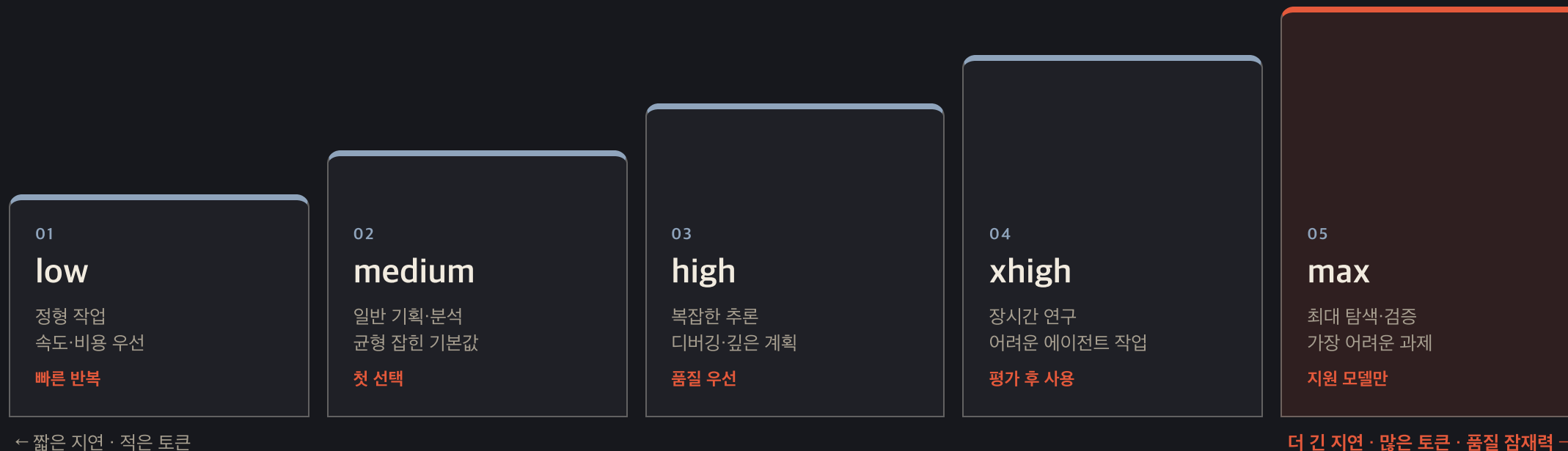
네 번 거르고, 마지막은 내 앱으로 시험합니다



리더보드는 후보를 좁히고, 내 과제가 최종 선발합니다.

low → medium → high → xhigh → max

얼마나 많은 추론 자원을 쓸지 정합니다



표기 정정: middle·row가 아니라 **medium·low**. 지원 단계와 기본값은 모델·플랫폼마다 다릅니다. — OpenAI reasoning effort ↗ · Anthropic effort ↗

max·xhigh와 N개 병렬 작업은 같은 말이 아닙니다

REASONING EFFORT

한 실행의 생각 강도

- 추론에 쓰는 시간·토큰을 조절
- 높을수록 더 탐색하고 검증할 수 있음
- 내부 구현 방식은 공개 문서 밖에서 추정하지 않음

≠

MULTI-AGENT · ORCHESTRATION

여러 실행의 작업 구조

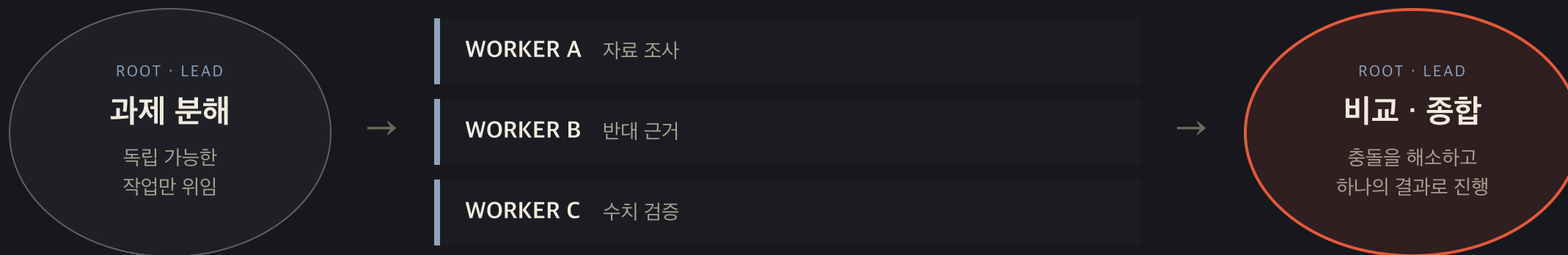
- 루트가 독립 과제를 N개로 나눔
- 각 작업자가 별도 문맥에서 병렬 수행
- 루트가 결과를 비교·종합해 다음 행동 결정

Thinking은 모델 설정, 병렬 실행과 종합은 플랫폼·하네스의 오케스트레이션입니다.

공개 문서의 선: GPT Pro는 “더 많은 model work 뒤 하나의 final answer”까지만 공개하며 best-of-N 구현이라고 설명하지 않습니다. Codex Ultra는 최대 추론과 선제적 subagent 위임을 함께 제공합니다. — [OpenAI Reasoning mode ↗](#) · [Codex Subagents ↗](#)

■ 공개된 병렬 구조

N개의 병렬 실행을 열고, 각 결과를 비교·종합해 진행합니다



병렬화는 넓은 조사와 분리된 검증에 유리하지만, **토큰·비용은 더 듭니다.**

공개 사례: OpenAI Multi-agent beta·일부 Codex Ultra 동작, Anthropic Research·subagent orchestration — OpenAI Multi-agent ↗ · Anthropic Multi-agent Research ↗

이름은 비슷해도, 조절 범위와 기본값은 다릅니다

공개 항목	GPT 계열	Claude Sonnet 5 · Opus 4.8
지원 단계	none · minimal · low · medium · high · xhigh · max 모델별 상이	low · medium · high · xhigh · max
문서상 기본값	medium 지원 모델·제품 설정에 따라 달라짐	high 현재 Sonnet 5 · Opus 4.8
effort가 조절하는 것	추론량 · 탐색 · 검증의 정도	추론뿐 아니라 출력·도구 호출까지 포함한 총 응답 토큰 지출
병렬 작업	Multi-agent·Ultra 등 별도 오케스트레이션	Research·subagent tools 등 별도 오케스트레이션
선택 원칙	medium에서 시작해 실제 평가로 올림	xhigh·max는 어려운 장기 과제에서 평가 후 사용

공통점: 높은 단계는 “무조건 더 정확함”이라는 보장이 아닙니다. 시간·토큰·비용과 실제 업무 평가를 함께 봅니다.

■ 그런데

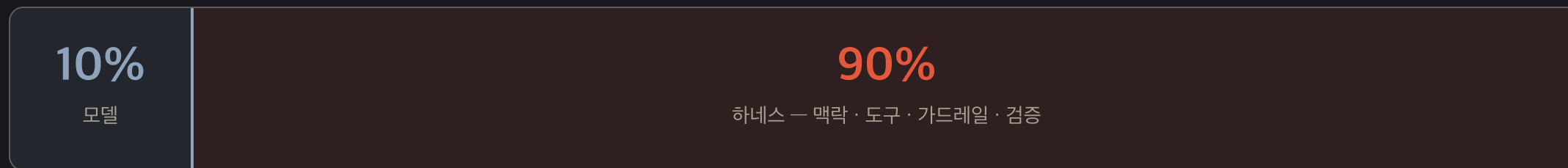
같은 Claude 모델을 골라도, 왜 채팅과 Claude Code의 결과는 다를까요?

엔진은 같아도 도구 · 권한 · 반복 · 기억을 묶는 방식이 다르기 때문입니다.

이 보이지 않는 절반을 하네스라고 부릅니다.

■ 용어 확인

하네스는 이미 **업계가 함께 쓰는** 말입니다



Google

OpenAI

Microsoft

Anthropic

Salesforce

Databricks

Red Hat

arXiv 논문 다수

같은 모델로 **하네스만 바꿔** 코딩 에이전트를 30위권 밖에서 5위권으로 올린 사례가 보고됐습니다.

비율은 Google 백서 *The New SDLC With Vibe Coding*(Osmani·Saboo·Kartakis)의 표현. 순위 사례는 Terminal Bench 2.0, LangChain은 같은 벤치마크에서 시스템 프롬프트·도구·미들웨어만 바꿔 13.7점 상승 — addyosmani.com ↗

모델이 좋아질수록, 병목은 모델 밖으로 이동합니다

THROUGHPUT

생성 속도는 급증

코드·문서·테스트를 만드는 속도는 사람의 작성 속도를 넘어섭니다.

SCARCE RESOURCE

사람의 주의를 고정

무엇을 만들지 정하고, 결과를 검수하는 시간은 늘지 않습니다.

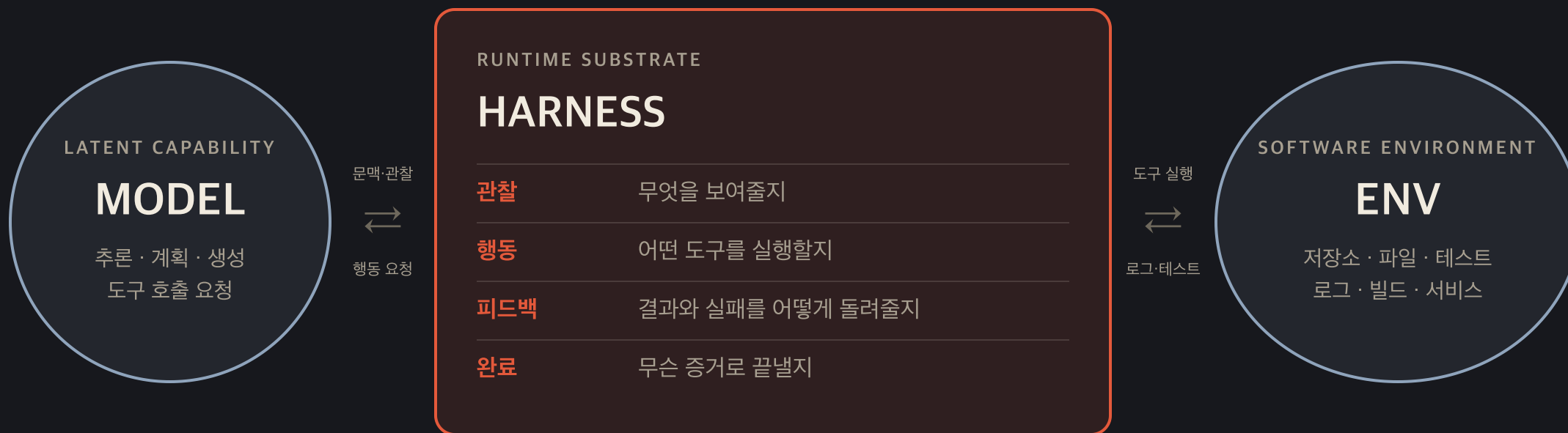
NEW BOTTLENECK

환경이 성능을 제한

맥락·도구·검증·피드백이 없으면 좋은 모델도 긴 일을 끝내지 못합니다.

그래서 플랫폼·개발팀의 일은 **환경을 설계하고 · 의도를 명시하고 · 피드백 루프를 만드는 일**로 올라갑니다.

하네스는 도구 상자가 아니라, 런타임 중개층입니다



하네스가 모델과 환경 사이에서 **관찰 → 행동 → 피드백 → 완료 판정**을 운영합니다.

플랫폼·개발팀이 관리하는 책임은 열한 가지입니다

01 · CONTRACT

목표 계약

과제 명세 · 성공 기준 · 제약

02 · CONTEXT

맥락과 상태

컨텍스트 선택 · 프로젝트 기억 · 과제 상태

03 · ACTION

행동 경계

도구 접근 · 권한과 승인

04 · SENSE & DIAGNOSE

감각과 진단

관찰 가능성 · 실패 원인 분류

05 · ASSURE & MAINTAIN

보증과 유지

검증 · 엔트로피 감사 · 사람 개입 기록

좋은 하네스는 답만 내지 않습니다. 왜 그렇게 했고, 무엇으로 검증했는지 남깁니다.

자율성은 프롬프트 길이가 아니라, 런타임 지원이 쌓이면서 올라갑니다

H0 · MINIMAL

과제 + 저장소

설명과 파일만 제공

H1 · TOOL

행동 가능

도구 목록
테스트 명령
사용 규칙

H2 · CONTEXT

길을 잃지 않음

프로젝트 기억
과제 상태
맥락 선택 규칙

H3 · VERIFY

완료를 증명

로그·런타임 관찰
버그 재현·원인 분류
결정적 검사
검증 보고서

주장이 아니라 증거

■ 여기서 역할이 바뀝니다

여기까지는 플랫폼의 런타임. 지금부터는 여러분의 업무 설계입니다

A · RUNTIME HARNESS

이해할 영역

루프 · 문맥 · 상태 관리
Claude Code · Codex 등 플랫폼

도구 실행 · 샌드박스
개발팀 · 관리자

재시도 · 복구 · 로그
직접 구현하지 않음

×

B · WORK DESIGN LAYER*

직접 설계할 영역

지도 · Skill · 업무 자료
무엇을 읽고 어떤 순서로 일할지

판단 기준 · 좋은 예시
무엇을 좋은 결과로 볼지

승인 · 예외 · 완료 조건
어디서 멈추고 사람에게 넘길지

모델 × 런타임 하네스 × 업무 설계층 — 세 요소가 함께 결과를 만듭니다.

* '하네스'는 업계에서 런타임 실행층을 가리키기도 하고(Salesforce·연구 논문), 지시문·Skill·검증을 담은 설계층을 가리키기도 합니다(Red Hat). 아직 한 단어로 정리되지 않아, 오늘은 런타임 하네스 / 업무 하네스로 나눠 부르겠습니다 — 강의용 구분입니다.

“다시 해봐” 대신, 무엇을 고칠지 업무 자산에 남깁니다



검수 의견을 대화에서 흘려보내지 않고, **다음 실행이 읽을 규칙**으로 바꿉니다.

■ B · 업무 하네스 · 부탁에서 계약으로

잘하라는 말 대신, 검수 가능한 업무 계약을 적습니다

PROMPT ONLY

“잘 검토해줘”

“중요한 것을 확인해줘”

무엇이 중요한지 모호함

“문제가 있으면 알려줘”

통과와 반려의 경계가 없음

“보기 좋게 정리해줘”

산출물 형식이 매번 달라짐



BUSINESS CONTRACT

기준을 글로 고정

확인 항목 · 필수 근거

무엇을 반드시 읽고 확인할지

통과 · 재작업 조건

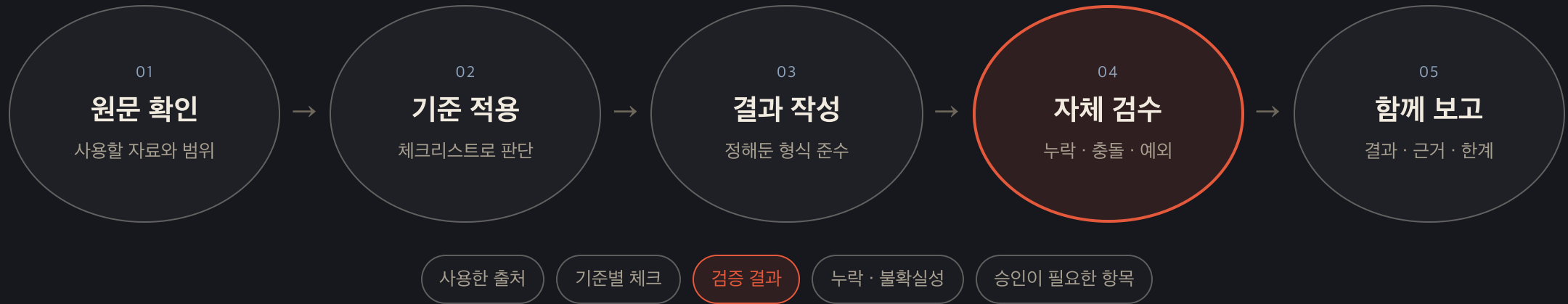
어떤 상태를 완료로 볼지

보고 형식 · 승인 경계

언제 멈추고 누구에게 물을지

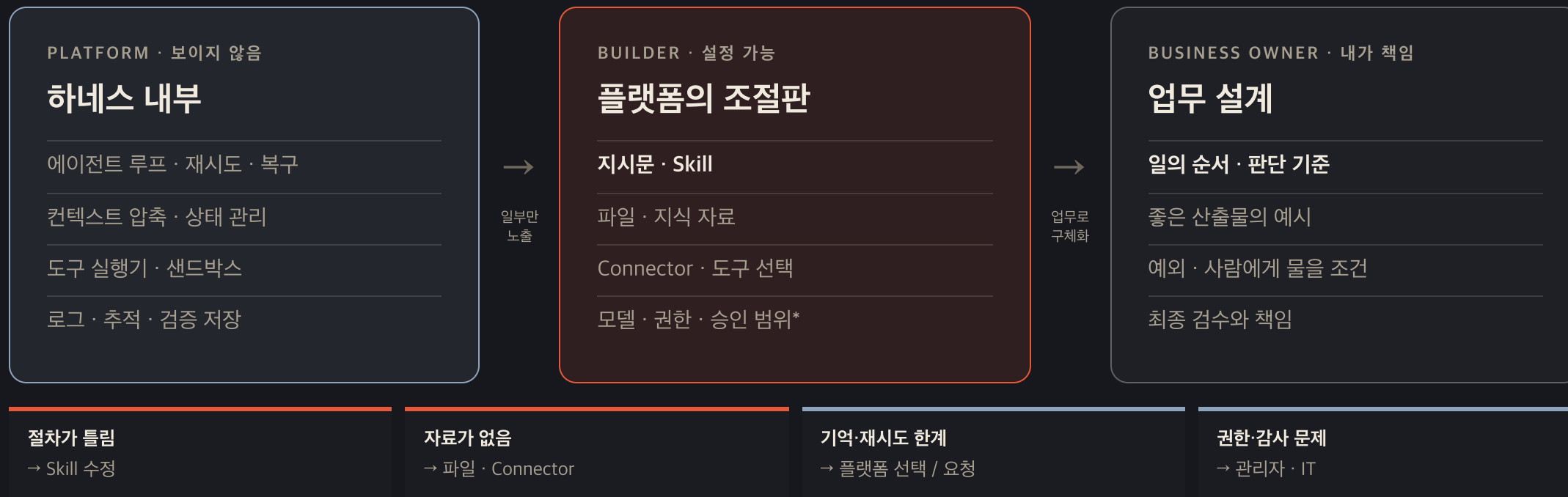
코드를 몰라도 **절차 · 판단 기준 · 중단 조건**은 설계할 수 있습니다.

완료는 “됐습니다”가 아니라, 결과와 근거를 함께 받는 것입니다



검사는 플랫폼이 실행해도, **무엇을 증거로 받을지는 업무 담당자가 정의합니다.**

비개발자는 Builder에서 하네스의 업무 설계층을 구성합니다



* 실제 노출 범위는 플랫폼과 조직 정책에 따라 다릅니다.

■ B · 업무 하네스 · 직접 만드는 최대 범위

런타임을 만들지 않아도, 업무 하네스는 직접 구성할 수 있습니다

01 · MAP

업무 지도 · 기억

claude_memory/MEMORY.md

gpt_memory/AGENTS.md

용어 · 결정 · 자료 위치 색인

02 · MATERIAL

업무 재료

SKILL.md · 실행 절차

원문 문서 · 데이터 · 링크

좋은 예시 · 템플릿

03 · CONTRACT

업무 계약

판단 기준 · 산출물 형식

승인 · 예외 · 중단 조건

사람에게 넘길 조건

코드로 루프를 만드는 대신, **모델이 찾아 읽고 그대로 일할 지도 · 재료 · 기준**을 만듭니다.

■ B · 업무 하네스 · 플랫폼에 요구할 것

내가 업무 자산을 만들면, 플랫폼은 여섯 가지를 보장해야 합니다

01 · LOAD

정확한 로딩

지도를 먼저 읽고, 관련 원문만 선택해 불러오기

02 · STATE

지속 가능한 상태

긴 작업 재개 · 작업 상태와 기억의 버전 관리

03 · ACCESS

연결과 최소 권한

필요한 도구만 허용 · 승인 · 즉시 회수

04 · VERIFY

검증 장치

기준별 검사 · 출처 · 완료 증거 묶음

05 · AUDIT

로그와 감사

사용 자료 · 도구 호출 · 변경 · 사람 개입 기록

06 · OWNERSHIP

소유권과 이동성

Skill · 기억 · 실행 기록을 내보내고 버전 관리

플랫폼·IT 요청 문장: “우리의 업무 자산을 **정확히 읽고 · 안전하게 실행하고 · 증거로 검증하고 · 전부 기록**해 주세요.”

이 앱을 만들며 쌓인 실제 작업 지도입니다

PROJECT FOLDER

claude_memory/

- ├ MEMORY.md
- ├ plan-execute-react...md
- ├ connector-frontend...md
- ├ no-hardcoded-text.md
- └ user-runs-rendering.md

MEMORY.md · INDEX

Memory Index

시스템 권한 및 아키텍처 원칙

백엔드 · 프론트엔드 작업 경계

Plan-Execute × ReAct 실행 모델

계획 · 실행 · 종료 계약의 정보 위치

Connector · 프론트 상태

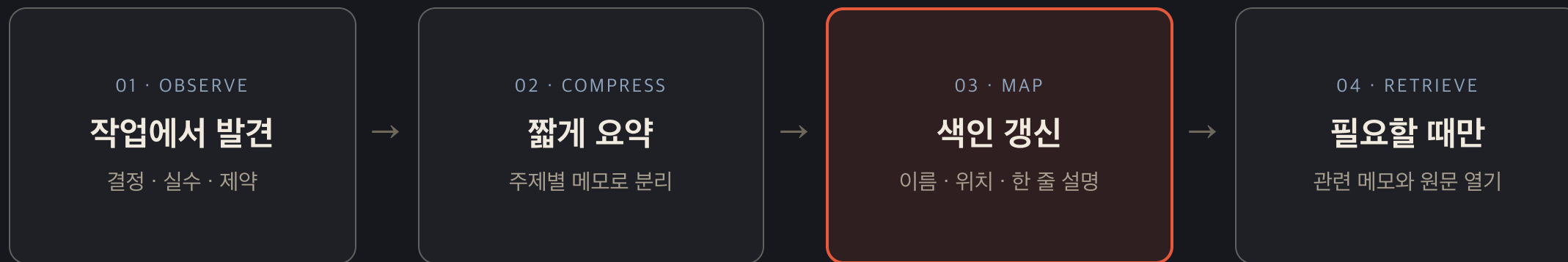
현재 연결 상태와 남은 제약

렌더링은 사용자가 직접

반복하지 말아야 할 작업 규칙

모든 내용을 복사한 창고가 아니라, 어디에 무엇이 있는지 알려주는 색인입니다.

플랫폼이 기억 기능을 제공하면, 모델이 일하면서 지도를 갱신합니다



하네스가 저장 자리와 불러오는 규칙을 제공하고, **모델이 요약과 색인을 작성**합니다.

Codex에는 **AGENTS.md**를 줍니다

강의용 샘플 보관

**gpt_memory/
AGENTS.md**

단수 agent.md가 아니라 복수형

AGENTS.md · PROJECT MAP

삼성생명 AI 교육 자료

먼저 볼 곳

메인 슬라이드 · 인트로 · 강의 뼈대 · 기억 색인

작업 원칙

관련 파일만 열기 · 사용자 변경 보존 · 1차 출처 확인

완료 기준

HTML 구조 · diff 검사 · 렌더링 역할

실사용 때 프로젝트 루트에 두면, Codex가 자동으로 불러오는 **에이전트용 프로젝트 README**가 됩니다.

RAG · 메모리 · 온톨로지에는 공통된 운영 패턴이 있습니다

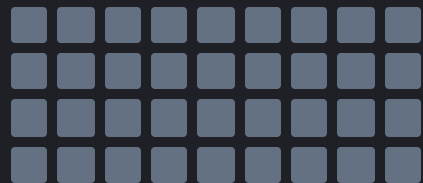
분야	먼저 보는 지도	필요할 때 펼치는 것
RAG	검색 질의 · 벡터 색인	관련 문서 조각
메모리	MEMORY.md 색인	관련 주제 메모
온톨로지	개념 · 관계 그래프	관련 엔터티와 규칙
코딩 에이전트	AGENTS.md · 프로젝트 지도	관련 코드 · 테스트 · 문서

전부 넣지 말고 — **찾을 수 있게 구조화한 뒤, 지금 필요한 것만 펼칩니다.**

단순한 Markdown이, attention을 아끼는 주소표가 됩니다

원문을 전부 넣기

모든 토큰이 경쟁

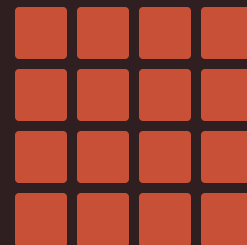


표준 self-attention의 계산량
 $O(n^2)$



지도 → 검색 → 원문

필요한 문맥만 활성화



비용 · 지연 · 잡음을 줄이고
중요한 단서에 집중

지도·검색이 유일한 수학적 해법은 아닙니다. 다만 모델 바깥에서 활성 문맥을 줄이는 대표적인 실무 패턴입니다. — Vaswani et al., 2017 ↗ · Lewis et al., 2020 ↗

■ 2교시 요약

“런타임 하네스는 플랫폼이,
업무 하네스는 내가.
나는 지도 · Skill · 자료 · 기준을 만들고,
로딩 · 실행 · 검증 · 감사를 요구합니다.”

■ 쉬는 시간 · 10분

10:00

다음 · 3교시 작업대 — Claude Desktop vs Claude Code

R 키 — 카운트다운 다시 시작

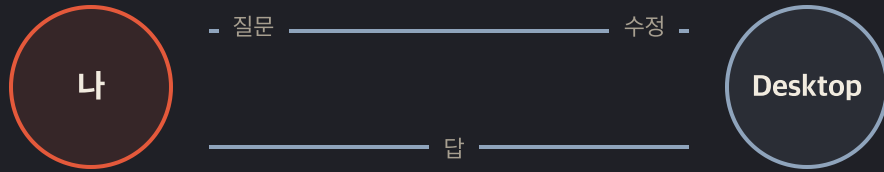
3

작업대

월요일 아침, 두 도구 중 무엇을 열지
3초 안에 판단하게 됩니다.

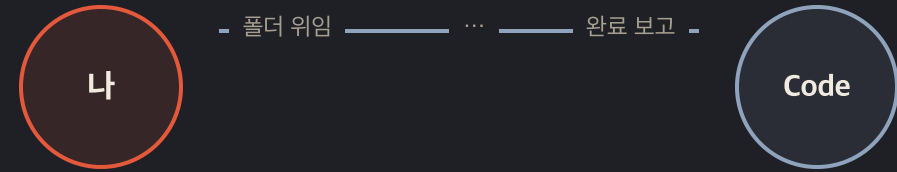
■ 본질적 차이

회의실에서 같이 일하기 vs 자리 내주고 위임하기



왕복하며 함께 다듬기

화면 · 대화 · 즉시 피드백



자리를 내주고 결과 받기

폴더 · 반복 · 여러 파일



결과가 **대화**면 Desktop, 결과가 **파일들**이면 Code.

■ 작업대 지형도

작업대가 다르면, 다른 도구입니다

ANTHROPIC · 화면

Claude Desktop

회의실의 대화 상대

파일 몇 개를 끌어다 놓고 왕복

Connector로 메일 · 일정 · 사내 시스템

결과를 보면서 바로 다듬기

잘 맞는 일

검토 · 초안 · 탐색

ANTHROPIC · 폴더

Claude Code

자리를 내준 동료

폴더를 통째로 맡기고 결과를 받음

파일을 직접 만들고 고침

Skill · Plugin · MCP가 모두 붙음

잘 맞는 일

대량 정리 · 반복 산출물

OPENAI

Codex

같은 문법을 쓰는 다른 팀

터미널과 클라우드 양쪽에서 위임

지시는 **AGENTS.md**로 — 2교시에서 본
그 파일

SKILL.md를 그대로 읽음

잘 맞는 일

GPT 생태계 안의 위임 작업

GOOGLE · MICROSOFT

Gemini CLI · Copilot

이미 깔려 있는 자리에서

검색 · 오피스 생태계와 가깝다

같은 SKILL.md 규격을 채택

회사 계정 정책을 먼저 확인할 것

잘 맞는 일

이미 쓰는 도구 옆에서

네 작업대가 모두 **같은 SKILL.md**를 읽습니다. 도구는 갈아탈 수 있고, 남는 것은 내 문서입니다.

GitHub는 2026-04 gh skill 명령을 공개했습니다 — 한 번의 설치가 Copilot · Claude Code · Cursor · Codex · Gemini CLI에 동시에 적용됩니다. [GitHub Docs](#) · [About agent skills](#) ↗

■ 고르는 기준

질문은 세 개면 충분합니다

결과물의 모양

01

내가 받고 싶은 것이 대화인가, 파일들인가.

→ 도구가 갑니다
Desktop인가 Code인가

자료의 경계

02

이 자료는 어디까지 나가도 되는가. 내 컴퓨터 안인가, 회사 시스템인가, 바깥 서비스인가.

→ 창구가 갑니다
어떤 Connector를 쓰는가

반복의 횟수

03

이번 한 번인가, 매주 돌아오는 일인가.

→ 남길 것이 갑니다
한 번이면 대화, 반복이면 Skill

이 세 질문이 오늘 남은 시간의 목차입니다 — 도구 · 창구 · 자산.

■ 3 초 판별법

결과물이 대화인가, 파일들인가

상황	도구	이유
아이디어 탐색 · 문서 1~3개 요약	Desktop	왕복 대화가 빠르다
보고서 20건 → 비교표 1개	Code	폴더 단위 위임
매주 반복하는 정형 산출물	Code + Skill	절차의 자산화
결과를 화면에서 보며 다듬기	Desktop	즉시 피드백

Code도 자연어로 씁니다. 명령어 암기 불필요 — 권한을 물어오면 확인하고 승인하는 습관이면 충분합니다.

■ 확장 문법

에이전트를 늘리는 것이 아니라, 하나를 **확장**합니다

지식을 더한다

Skill

내 업무의 절차와 기준을 문서로. 필요할 때 펼쳐 읽는다.

손이 닿는 범위를 늘린다

Connector · MCP

메일 · 일정 · 사내 시스템으로 나가는 표준 창구.

ALREADY THERE

에이전트

Claude Desktop · Claude Code

이미 설치되어 있는 실행 주체

팀으로 퍼뜨린다

Plugin

검증한 Skill · 창구 · 명령을 한 상자에 담아 배포한다.

손대지 않는 층

모델 · 런타임

2교시의 그 층. 고르기는 하되 만들지는 않는다.

에이전트 개수를 세는 대신, **무엇을 붙일지**를 정합니다.

연결마다 새로 만들지 않기 위한 **규격**

규격이 없다면

도구 4 × 시스템 5



20개의 개별 연동

도구가 하나 늘 때마다 5개씩 더



MCP라는 공통 규격

도구 4 + 시스템 5



9개의 표준 연결

한 번 만든 창구를 모든 도구가 쓴다

Connector는 **소비자용 이름**, MCP는 **그 규격의 이름**입니다 — 충전 단자를 통일한 것과 같습니다.

■ 창구는 세 종류

고르는 기준은 성능이 아니라 자료가 가는 거리입니다

① 공식 커넥터

제공사가 만든 것

Gmail · 캘린더 · 드라이브 등

로그인 승인(OAuth)으로 연결

내 계정 권한 밖은 보지 못한다

자료가 가는 거리 — 해당 서비스까지

② 원격 MCP

우리 회사가 만든 것

공식 커넥터가 없는 사내 시스템

읽기 전용 · 권한 범위를 IT가 통제

감사 로그를 남길 수 있는 구조로

자료가 가는 거리 — 우리 서버까지

③ 로컬 MCP

내 컴퓨터 안에서만

이 노트북 밖으로 나가지 않는다

네트워크가 끊겨도 동작한다

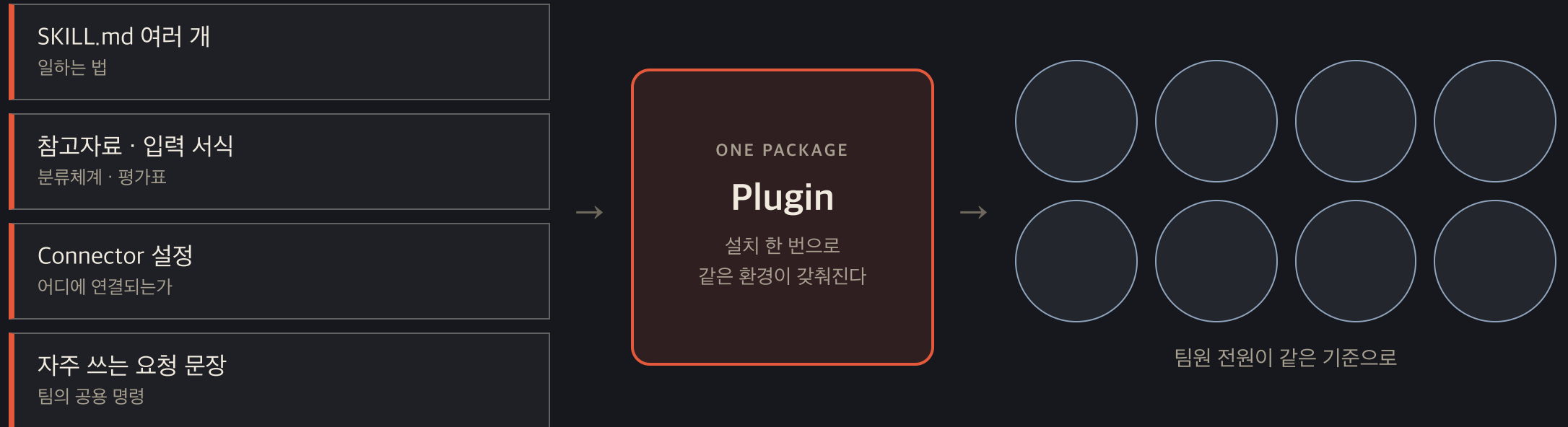
잠시 후 시연할 "맥 비서"가 이것

자료가 가는 거리 — 이 책상 위까지

붙이기 전 다섯 가지 — 누가 만들었나 · 무엇을 읽나 · 무엇을 쓰나 · 기록이 남나 · 어떻게 끝나.
금융사에서 신뢰는 붙이는 법이 아니라 끊는 법을 아는 데서 생깁니다.

■ Plugin

한 사람이 검증한 방식을 팀 전체가 한 번에



Plugin은 새로운 능력이 아니라 **배포 단위**입니다 — 팀 표준 업무 키트.

■ LIVE DEMO ①

같은 과제, 두 도구

과제: 샘플 문서 묶음으로 비교표 만들기

Desktop — 파일을 끌어 넣고 대화로. 즉각적입니다. 그런데 문서가 30개라면?

Code — 폴더를 주고 "전부 읽고 비교표 파일로 저장해줘".

파일이 생겨나는 순간을 보십시오.

창구를 붙이면, AI가 이 방에서 소리를 냅니다

1 · 붙인다

설정 → Connectors

"맥 비서" — 이 노트북 안에서만 도는 로컬 창구. 도구 다섯 개가 목록에 나타납니다.

2 · 시킨다

한 문장으로 네 가지

상품 DB 조회 → 40대 가장 관점 비교 →
결론을 소리 내어 읽기 → 알림 · 미리알림
등록

3 · 끊는다

연결 해제까지

붙이는 화면만 보여주고 끝내지 않습니다.
끊는 자리를 같은 화면에서 확인합니다.

가상 상품 데이터입니다. 실존 회사·상품과 무관하며, 응답에도 "가상 데이터" 표기가 붙습니다.

■ 일반화

여러분의 "연출 언어"는 여러분 업무 안에 이미 있습니다

보험 실무자가 배울 것은 프로그래밍이 아니라 —

좋은 심사보고서의 구조 · 좋은 비교표의 기준 · 좋은 요약의 조건.

그것을 문서로 쓰면, 그게 Skill입니다. 다음 시간에 직접 만듭니다.

■ 삼성생명 사전 자료와 연결

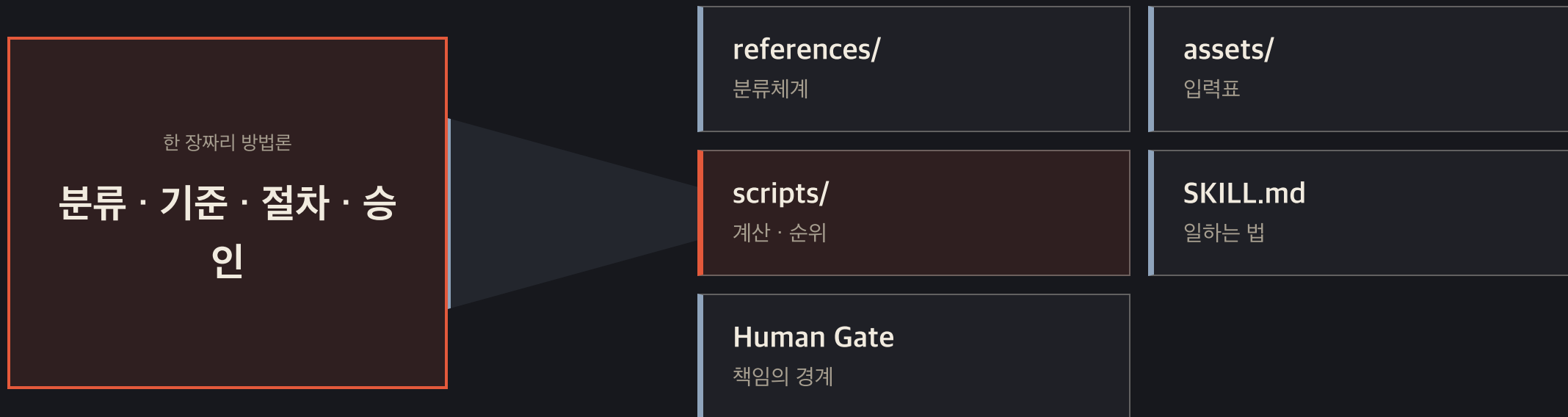
74 → 39 → 25 → 5 → 100+ → 22
이 숫자의 실체는 **업무 퍼널**입니다



에이전트 수보다 먼저 **입력 · 판단 · 승인 기준**을 고정합니다.

■ 방법론을 Skill로 번역하기

한 장짜리 설계도가 다섯 종류의 파일로 바뀝니다



설명은 사라지지 않습니다. **다시 실행 가능한 구조**로 분해됩니다.

전체 산업 대신 작은 mock으로, 같은 구조를 직접 돌립니다

공통 Skill 실행

후보 8개 → Short-list 5개

가상 생명보험사 데이터 · 실제 당사 결과 아님

내 기준으로 수정

평가기준 1개 + Human Gate 1개

수정 전후 순위와 판단 근거를 비교

Skill을 만든다는 것은 AI 역할 이름을 붙이는 일이 아니라,
우리 조직의 판단 기준을 실행 가능한 파일로 만드는 일입니다.

■ 3 교 시 요약

"대화면 Desktop, 파일 작업이면 Code.
지식은 Skill, 손이 닿는 범위는 Connector, 팀 배포는 Plugin.
그리고 내 업무의 연출 언어는
분류체계 · 평가기준 · Human Gate다."

■ 쉬는 시간 · 10분

10:00

다음 · 4교시 손 — 직접 만들어보는 시간 (노트북을 준비해주세요)

R 키 — 카운트다운 다시 시작

4

손

지금부터는 여러분 손이 움직입니다.

공통 Skill을 실행해 기준을 바꾸고, 창구를 직접 붙여봅니다.

■ 실습 ① · 10분

완성된 Skill부터 실행해봅시다

- discover-new-business-opportunities 폴더를 연다
- 가상 생명보험사 mock으로 Sector Short-list와 BM 순위를 만든다
- 결과 파일 2개를 확인한다 — JSON은 기계용, Markdown은 사람용
- 점수보다 먼저 근거 신뢰도와 Human Gate를 찾는다

실제 당사 데이터·평가 결과·고객정보는 사용하지 않습니다.

우리 팀의 판단 기준으로 Skill 바꾸기

- 7개 섹터 기준 중 우리 팀에 맞지 않는 기준 1개를 고른다
- 점수 또는 설명을 바꾸고 변경 이유를 한 줄로 적는다
- Human Gate 하나를 추가한다 — 누가, 무엇을, 언제 승인해야 하는가
- Skill을 다시 실행해 변경 전후 Short-list를 비교한다
- 마지막 5분 — 이 구조를 최근 반복 업무 하나에 복제한다

■ 복사해서 쓰세요

요청 문장

"\$discover-new-business-opportunities로 워크숍 mock을 실행해줘.
우리 팀에 맞게 평가기준 하나와 Human Gate 하나를 바꾸고,
변경 전후 순위와 근거를 비교해줘. 실제 데이터는 쓰지 마."

성공 기준 — 설명을 길게 쓰는 것이 아니라, 기준을 바꾸면 결과가 재현 가능하게 달라지는 것.

■ 실습 ③ · 10분

이번엔 공식 커넥터를 직접 붙여봅니다

- 3교시의 세 종류 중 ① 공식 커넥터 — 설정 → Connectors → Gmail → 로그인 승인
- 과제: "지난 2주 메일 중 아직 회신 안 한 것 같은 메일을 찾아 3줄씩 요약해줘"
- 그리고 반드시 — 연결 해제하는 법, 권한 확인하는 법

AI는 내 권한 이상은 절대 못 봅니다. 금융사에서는 붙이는 법보다 끊는 법을 아는 게 신뢰를 삽니다.

■ 실습 회고 · 10분

처음 받은 설계도와, 방금 만든 Skill을 겹쳐봅니다

신사업추진파트 — "Agent 활용 신사업 발굴 방법론" 도식

- 역할별 에이전트의 입력·기준·산출물 계약 — 문서로 남는 Skill
- 오케스트레이터와 루프 — 빌려 쓰는 층, 플랫폼이 가장 잘하는 부분
- 단계별 사람 게이트 4개 — 자동화할수록 더 선명해야 하는 책임선

■ 오늘의 네 문장

가지고 가실 것

- 에이전트는 이미 준비되어 있다 — 내가 만들 것은 Skill이다
- 런타임 하네스는 플랫폼에 요구하고, 업무 하네스는 지도 · Skill · 자료 · 기준으로 직접 설계한다
- 대화면 Desktop, 파일 작업이면 Code
- 배워야 할 것은 구현 기술이 아니라 내 업무의 연출 언어다

■ Next Action

다음 주에 Skill을 **딱 하나**만 더 만들어보세요.

그 문서 파일이, 어느 플랫폼을 쓰든 들고 다닐 여러분의 자산입니다.

오프닝의 그 화면을 만든 건 기술이 아니라 연출 언어였습니다.

여러분의 연출 언어는 — 여러분 업무 안에 이미 있습니다.